

Extending Kubernetes Self-Healing for Networking Failures

CS331 - Computer Networks Project

Arpan Gupta (24110051)

Buha Deep Maheshbhai (24110082)

Param Tanna (24110236)

Ramji Purwar (24110287)

Rathod Yuvraj Rajubhai (24110293)

Solanki Viraj Rajeshbhai (24110348)

Problem Statement & Objective

Problem: Kubernetes heals container crashes but is **blind to network-layer failures**.

- ▶ CNI plugin crashes → pods stuck in ContainerCreating
- ▶ CoreDNS degradation → silent DNS resolution failures
- ▶ Overlay tunnel drops → cross-node traffic silently fails
- ▶ Pods show Running (1/1 Ready) while network is dead

Objective

Build a **custom Kubernetes operator** that continuously monitors and auto-remediates three critical network subsystems:

1. **CNI plugin health** (Calico IPAM exhaustion & agent failures)
2. **CoreDNS availability** (scale-to-zero, corrupted upstreams, latency spikes)
3. **Pod-to-pod connectivity** (overlay tunnel drops & cross-node routing)

Limitation of Kubernetes

CNI Plugin (Calico)

- ▶ calico-node agent crash
- ▶ IPAM pool exhaustion
- ▶ Disabled IPPool

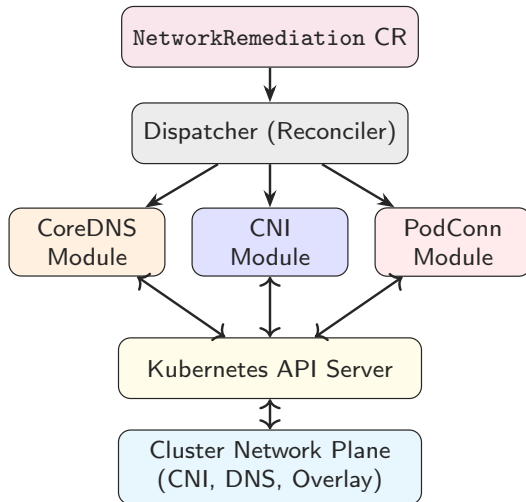
CoreDNS

- ▶ Replicas scaled to zero
- ▶ CPU throttling (≤ 5 m)
- ▶ Latency spikes (> 1000 ms)
- ▶ Corrupted upstream forward

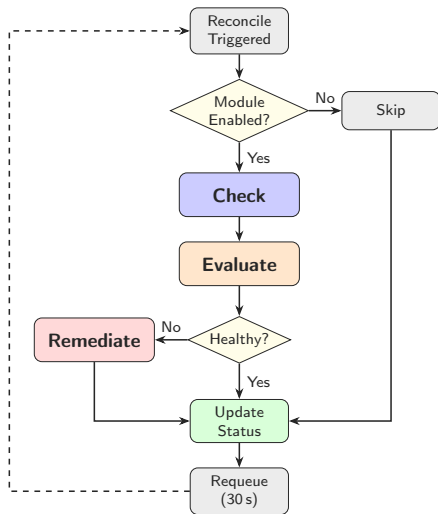
Pod-to-Pod

- ▶ Virtual interface (veth) down
- ▶ Overlay tunnel (IPIP) drop

System Architecture



Three-Phase Pipeline: Check → Evaluate → Remediate



Check - Signal Gathering

Collects raw network telemetry: pod statuses, DaemonSet health, IPAM allocations, DNS latency, active probe results. *No decisions made.*

Evaluate - Failure Classification

Analyzes signals against configurable thresholds. Classifies severity: info, warning, critical. Prevents false positives.

Remediate - Corrective Action

Executes minimum K8s API mutations: pod deletions, deployment scaling, ConfigMap patches, node taints, workload evictions.

CRD-Driven Configuration

Module	Key Parameter	Default
CNI	stuckPodThresholdSec	60 s
	ipamUsageThreshold%	80%
	calicoNodeLabel	calico-node
CoreDNS	expectedReplicas	2
	latencyThresholdMs	150 ms
	autoHealReplicas	true
	autoHealConfigMap	true
Pod Conn	consecutiveFailThreshold	3
	autoRemediationEnabled	true
	maxRestartAttempts	2

Key Design Points:

- ▶ Single **cluster-scoped** CRD: NetworkRemediation (v1alpha1)
- ▶ Each module independently **toggleable** via enabled field
- ▶ Safety guardrails: cooldown windows, escalation limits, failure thresholds
- ▶ Supports **audit-only mode**: observe and log without mutating

CNI Module: Detection & Remediation

Detection Signals:

1. **DaemonSet health:**
desiredNumberScheduled vs numberReady
2. **Stuck workload pods:** Pending phase + empty podIP exceeding threshold
3. **CNI event correlation:**
FailedCreatePodSandBox warning events with CNI/IPAM signatures
4. **IPAM capacity:** Calico IPPool and IPAMBlock CRDs for disabled pools and exhausted blocks

Evaluation Priority Order:

1. Unready calico-node → critical, restart agents
2. Disabled IPPool + stuck pods → critical, re-enable pool
3. Stuck pods + CNI errors → critical, evict pods
4. IPAM usage $\geq$ threshold → warning, log only

Remediation Actions:

- **Restart:** Delete unready calico-node pods
- **Re-enable:** Patch spec.disabled → false
- **Evict:** Delete stranded pods for rescheduling

CoreDNS Module: Detection & Remediation

Detection Signals:

1. **Replica availability:** `availableReplicas` vs `expectedReplicas`
2. **Pod readiness:** `CrashLoopBackOff`, `Error`, or `unready` states
3. **DNS query latency:** via Prometheus metrics

$$\text{Latency} = \frac{\text{rate}(\text{sum}[1\text{m}])}{\text{rate}(\text{count}[1\text{m}])}$$

4. **Upstream validation:** Corefile forward targets checked against RFC 5737 bad subnets
5. **Resource starvation:** CPU limits ≤ 5 m causing latency spikes

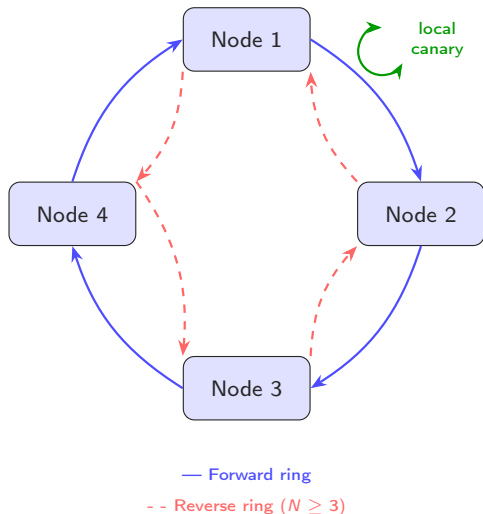
Remediation Actions:

- ▶ **Scale recovery:** Patch replicas back to `expectedReplicas`
- ▶ **Pod restart:** Delete `unready/crashlooping` pods
- ▶ **ConfigMap repair:** Replace corrupted forward targets with fallbacks (`/etc/resolv.conf`, 8.8.8.8)
- ▶ **Resource restoration:** Reset CPU limits to 100 m baseline

Safety Mechanisms:

- ▶ Independent toggles: `autoHealReplicas`, `autoHealConfigMap`
- ▶ 30 s cooldown window per remediation type

Pod Connectivity Module: $O(N)$ Pingmesh Ring Probing



Topology Design:

- ▶ Nodes sorted lexicographically, connected in a ring
- ▶ Forward: $Node_i \rightarrow Node_{(i+1) \bmod N}$
- ▶ Reverse: $Node_i \rightarrow Node_{(i-1+N) \bmod N}$

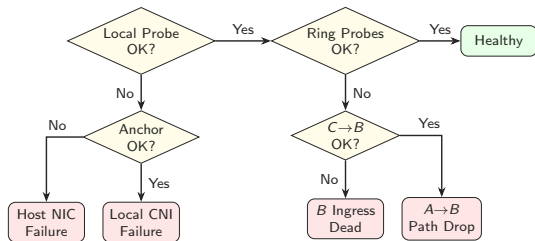
Scalability:

- ▶ Exactly $2N$ inter-node probes $\rightarrow O(N)$
- ▶ vs. full-mesh probing: $O(N^2)$
- ▶ + $O(1)$ intra-node canary per node (veth, bridge, iptables FORWARD chain)

Adapted from:

Microsoft Pingmesh [SIGCOMM 2015]

Three-Tier Triangulation for Root-Cause Isolation



Tier 1 - Local CNI Validation:

Local probe fails but anchors (gateway, CoreDNS) reachable → local CNI issue

Tier 2 - Inter-Node Triangulation:

$A \nrightarrow B$: check if $C \rightarrow B$. All peers fail → B ingress dead. Only A fails → asymmetric path drop

Tier 3 - Anchor Corroboration:

For small clusters ($N < 3$): probes against gateway, kube-apiserver disambiguate host vs tunnel failures

Hierarchical Remediation:

1. CNI Agent Restart (max 2 attempts)
2. Node Taint & Cordon (NoSchedule)
3. Workload Drain (Eviction API)

Experimental Setup

Cluster Environment:

Component	Configuration
Cluster	Minikube (Docker), 2 nodes
Resources/Node	2 vCPUs, 4096 MB RAM
CNI Plugin	Calico (DaemonSet)
DNS Server	CoreDNS (2 replicas)
K8s Version	v1.31+
Operator	Kubebuilder v4.15+, Go 1.24+

Fault-Injection Methodology:

1. Establish **healthy cluster baseline**
2. **Inject** specific network fault; observe native K8s behavior (no operator)
3. **Enable operator**; verify automated detection and recovery

Test Workloads:

- ▶ backend-api - HTTP service (2 replicas, spread across nodes)
- ▶ frontend-probe - continuous HTTP poller
- ▶ dns-checker - DNS resolution verifier

Conclusion & Future Work

Conclusion:

- ▶ Extends K8s self-healing to the **network data plane**
- ▶ Modular **Check** → **Evaluate** → **Remediate** pipeline
- ▶ $O(N)$ **Pingmesh ring probing** + 3-tier triangulation
- ▶ Validated across **8/8 fault experiments** (30–60 s recovery)

Future Work:

- ▶ **Alertmanager:** Slack, PagerDuty notifications
- ▶ **eBPF Probing:** Kernel-level introspection with lower overhead
- ▶ **ML Anomaly Detection:** Time-series DNS & latency models
- ▶ **Multi-Cluster:** Cross-cluster network mesh healing

Thank You!

Arpan Gupta (24110051)

Buha Deep Maheshbhai (24110082)

Param Tanna (24110236)

Ramji Purwar (24110287)

Rathod Yuvraj Rajubhai (24110293)

Solanki Viraj Rajeshbhai (24110348)